

Technische Projektdokumentation

Mensa Web-Applikation

Dokumentation der Änderungen seit GitHub-Initialisierung

Autor: Ingo Weber (ITSE2501)

7. Mai 2026

Inhaltsverzeichnis

1	Einleitung	3
2	Analyse des Ist-Zustands	4
2.1	Strukturelle Mängel	4
2.2	Sicherheitsrelevante Schwachstellen	4
2.3	Benutzererfahrung und Design	4
3	Systemarchitektur	5
3.1	Technologie-Stack	5
3.2	Verzeichnisstruktur (Vergleich)	5
4	Backend-Implementierung	6
4.1	Modernisierung der Datenbankschicht	6
4.1.1	Datenbankverbindung (Singleton-Pattern)	6
4.2	Einführung eines Template-Systems	6
4.3	Sicherheitskonzept	6
5	Datenbankmodell	7
6	Frontend & Design-System	8
6.1	Visual Excellence & Design-Tokens	8
6.2	Responsive App-Experience	8
7	Wichtige Änderungen (Refactoring)	9
8	Lizenz	10
A	Technische Code-Beispiele	11
A.1	Vergleich: Datenbank-Anbindung (Alt vs. Neu)	11
A.1.1	Ist-Zustand (Alt: conn.php)	11
A.1.2	Soll-Zustand (Neu: Database.php)	11
A.2	Datenbank-Singleton (Database.php)	11
A.3	Ticket-System Algorithmus (index.php)	12
A.4	Ergebnisvisulierung (Logik)	13
A.5	Design-Variablen (CSS)	13

1 Einleitung

Dieses Dokument beschreibt die technische Architektur und die Implementierung der Mensa Web-Applikation. Schwerpunkt dieser Dokumentation sind die umfangreichen Modernisierungs- und Refactoring-Maßnahmen, die seit der ersten Bereitstellung auf GitHub durchgeführt wurden.

Das Projekt dient der Verwaltung von Speiseplänen, der Durchführung von Abstimmungen über Wunschgerichte und der Kommunikation zwischen Nutzern und dem Küchenteam. Ziel der Überarbeitung war es, eine technisch veraltete und schwer wartbare Codebasis in eine moderne, sichere und benutzerfreundliche Anwendung zu transformieren.

2 Analyse des Ist-Zustands

Vor der Modernisierung wies die Applikation (dokumentiert im Verzeichnis `mensa.old`) signifikante technische und gestalterische Defizite auf, die eine Weiterentwicklung erschwerten.

2.1 Strukturelle Mängel

Die ursprüngliche Codebasis war monolithisch und wenig strukturiert:

- **Dateiorganisation:** Nahezu alle PHP-Skripte, SQL-Dumps und Hilfsdateien lagen unsortiert im Hauptverzeichnis. Es gab keine klare Trennung zwischen Logik, Konfiguration und Darstellung.
- **Spaghetti-Code:** PHP-Logik und HTML-Markup waren eng miteinander verzahnt. Datenbankabfragen wurden direkt innerhalb der Darstellungsschicht ausgeführt, oft redundant durch mehrfaches Einbinden der `conn.php`.
- **Code-Duplikation:** Ähnliche Funktionalitäten (z. B. Datenbankverbindungen oder Header-Elemente) wurden in vielen Dateien separat implementiert oder über unflexible `require`-Ketten geladen.

2.2 Sicherheitsrelevante Schwachstellen

Die Sicherheit entsprach nicht modernen Standards:

- **Datenbank-Anbindung:** Zugangsdaten waren im Klartext in der `conn.php` hartkodiert. Es gab kein zentrales Konfigurationsmanagement für unterschiedliche Umgebungen (Lokal vs. Produktion).
- **Authentifizierung:** Das Admin-System basierte auf einfacheren Mechanismen ohne durchgängige Nutzung moderner Hashing-Verfahren für alle Bereiche.
- **Eingabevalidierung:** Der Schutz gegen SQL-Injection war nicht flächendeckend durch Prepared Statements gewährleistet, was ein potenzielles Sicherheitsrisiko darstellte.

2.3 Benutzererfahrung und Design

Das Frontend war funktional, aber optisch veraltet:

- **UI-Framework:** Die Nutzung von W3.CSS erfolgte ohne Anpassungen oder ein konsistentes Design-System. Die Oberfläche wirkte generisch und wenig professionell.
- **Ansprache:** Die Nutzer wurden durchgängig mit 'Du' angesprochen, was für eine offizielle Einrichtung wie eine Mensa als zu informell eingestuft wurde.
- **Responsive Design:** Zwar war die Seite grundsätzlich responsiv, jedoch fehlte eine gezielte Optimierung für moderne mobile Endgeräte (z. B. App-ähnliche Navigation, Full-Screen-Layouts).

3 Systemarchitektur

Die neue Architektur zielt auf eine saubere Trennung von Belangen (Separation of Concerns) ab, um die Wartbarkeit und Erweiterbarkeit zu erhöhen.

3.1 Technologie-Stack

Die Applikation basiert auf einem klassischen, aber modernisierten LAMP/WAMP-Stack. Der Fokus lag hierbei auf der Konsolidierung der Backend-Logik:

- **Backend:** PHP 8.x. Übergang von rein prozeduralem Code zu einem objektorientierten Ansatz für Kernkomponenten (Datenbank, Session-Handling).
- **Datenbank:** MariaDB / MySQL. Migration von der veralteten `mysql_*`-Logik (in sehr alten Versionen) bzw. unstrukturiertem PDO hin zu einem zentralen PDO-Singleton.
- **Frontend:** HTML5, CSS3 (Custom Design System), JavaScript (Vanilla). Verzicht auf schwere Bibliotheken zugunsten von Performance und Kontrolle.
- **Styling-Basis:** Modifiziertes W3.CSS als Grid-System, ergänzt durch ein umfangreiches Set an CSS-Custom-Properties (Variables).

3.2 Verzeichnisstruktur (Vergleich)

Ein wesentlicher Teil des Refactorings war die Reorganisation der Dateien.

Tabelle 3.1: Vergleich der Verzeichnisstrukturen

Komponente	Alte Struktur (Ist)	Neue Struktur (Soll)
Konfiguration	<code>conn.php</code> (im Root)	<code>/config/config.php</code>
Datenbank-Logik	Verteilt in PHP-Dateien	<code>/includes/Database.php</code>
UI-Komponenten	<code>block*.php</code> (im Root)	<code>/templates/*.php</code>
Dokumentation	Nicht vorhanden / Lokal	<code>/docs/*.tex</code>
SQL-Skripte	Viele Dumps im Root	<code>/mesa.sql</code> (bereinigt)
Bilder	<code>/images</code>	<code>/images</code> (strukturiert)

Durch diese Modularisierung wurde sichergestellt, dass Änderungen an der Datenbankverbindung nur an einer einzigen Stelle (`config.php`) vorgenommen werden müssen, anstatt in jeder PHP-Datei einzeln.

4 Backend-Implementierung

4.1 Modernisierung der Datenbankschicht

Im alten System wurde die Verbindung über eine globale Variable `$conn` in der Datei `conn.php` realisiert. Diese Datei wurde in fast jedem Skript mittels `require` eingebunden, was zu unübersichtlichen Abhängigkeiten und Problemen bei Unit-Tests oder der Skalierung führte.

4.1.1 Datenbankverbindung (Singleton-Pattern)

Die Datenbankverbindung wurde in der Klasse `Database.php` zentralisiert. Dies stellt sicher, dass pro Request nur eine Verbindung aufgebaut wird (Singleton). Zudem werden Konfigurationsparameter nun aus einer separaten `config.php` geladen, die außerhalb des öffentlichen Web-Verzeichnisses liegen kann.

4.2 Einführung eines Template-Systems

Um Redundanz zu vermeiden, wurden Kopf- und Fußbereiche in Templates ausgelagert (`header.php`, `footer.php`). Das neue System nutzt PHP-Variablen zur Steuerung von Seitentiteln und aktiven Menüpunkten, was die Logik in den einzelnen Inhaltsseiten (z. B. `index.php`, `login.php`) drastisch reduziert hat.

4.3 Sicherheitskonzept

Ein wesentlicher Teil des Projekts war die Behebung der im Ist-Zustand identifizierten Sicherheitslücken.

- **Zentrales Konfigurationsmanagement:** Zugangsdaten werden nicht mehr in der Verbindungsdatei hartkodiert, sondern über Konstanten in der `config.php` verwaltet.
- **Passwort-Hashing:** Admin-Passwörter werden mittels `password_hash()` und dem BCRYPT-Algorithmus sicher gespeichert. Alte Klartext- oder schwach gehashte Passwörter wurden im Zuge der Migration ersetzt.
- **Ticket-Sicherheit:** Für Rückantworten im Kontaktformular wird ein Geheimwort-Hashing eingesetzt. Das System generiert eine Ticket-ID und ein zufälliges Wort, das nur dem Einreicher bekannt ist.
- **SQL-Injection Schutz:** Durch den Einsatz von PDO und consequenten *Prepared Statements* wurde die Anfälligkeit für SQL-Injektionen eliminiert.

5 Datenbankmodell

Das Schema umfasst mehrere Tabellen zur Abbildung der Geschäftslogik:

1. **speisen:** Stammdaten aller verfügbaren Gerichte.
2. **wochenplan** / **wunschplan:** Zuordnung von Speisen zu Kalendertagen.
3. **nachrichten:** Speicherung von Feedback und Support-Tickets inkl. Status (Antwort/Rückantwort).
4. **custom_surveys:** Dynamische Umfragen mit flexiblen Fragen und Antwortmöglichkeiten.
5. **abstimmung:** Zählung der Stimmen für die wöchentliche Wahl.

6 Frontend & Design-System

Das neue Design-System bricht mit dem generischen Look des ursprünglichen W3.CSS-Einsatzes.

6.1 Visual Excellence & Design-Tokens

Statt fester Farbklassen werden nun CSS-Variablen (*Design Tokens*) verwendet. Dies ermöglicht eine einfache Anpassung des gesamten Erscheinungsbildes (z. B. für saisonale Aktionen) an einer zentralen Stelle in der `style.css`.

- **Farbpalette:** Ein moderner Dark Mode mit tiefen Blautönen (`#0B0E14`) und einer klaren Akzentfarbe (`#3B82F6`) sorgt für eine hochwertige Anmutung.
- **Glassmorphismus:** Karten und Overlays nutzen `backdrop-filter` und subtile Transparenzen, um Tiefe zu erzeugen.

6.2 Responsive App-Experience

Besonderes Augenmerk wurde auf die mobile Nutzung gelegt:

- **Tab-Navigation:** Eine am unteren Bildschirmrand fixierte Navigationsleiste (ähnlich einer nativen App) erleichtert die Bedienung mit einer Hand.
- **Content-Management:** Große Tabellen aus dem Ist-Zustand wurden in der Mobilansicht durch stapelbare Karten ersetzt, um horizontales Scrollen zu vermeiden.

7 Wichtige Änderungen (Refactoring)

Förmliche Ansprache (Sie): Die gesamte Anwendung wurde von der 'Du'-Form auf die formelle SSie-Ansprache umgestellt. Dies betrifft nicht nur Texte in der UI, sondern auch Fehlermeldungen und Systembenachrichtigungen.

Zusammenführung der Logik: Statt für jede Funktion (Abstimmung, Umfrage, Kontakt) eine eigene PHP-Datei aufzurufen, wird nun ein zentraler Controller-Ansatz in der `index.php` genutzt. Dies reduziert die Ladezeiten und verbessert die User Experience.

Fehlerbehandlung: Einführung einer zentralen Fehlerbehandlung, die dem Nutzer verständliche Rückmeldungen gibt, anstatt kryptische PHP-Fehlermeldungen anzuzeigen.

Mobil-Optimierung: Integration von Viewport-Anpassungen für Geräte mit Display-Aussparungen (Notch) und Optimierung der Touch-Targets.

8 Lizenz

Das Projekt steht unter der **GNU General Public License v3.0**. Dies erlaubt die freie Nutzung und Modifikation unter Beibehaltung der Lizenzbedingungen.

A Technische Code-Beispiele

In diesem Anhang werden Ausschnitte der zentralen Logik-Komponenten aufgeführt, die im Rahmen der Modernisierung implementiert wurden.

A.1 Vergleich: Datenbank-Anbindung (Alt vs. Neu)

Hier wird der strukturelle Unterschied zwischen der alten prozeduralen Einbindung und dem neuen objektorientierten Singleton-Ansatz deutlich.

A.1.1 Ist-Zustand (Alt: conn.php)

```
1 // conn.php - Wurde in jeder Datei inkludiert
2 $server = "localhost";
3 $user = "root";
4 $passwd = "password";
5 try {
6     $conn = new PDO("mysql:host=$server;dbname=mensa", $user, $passwd);
7     $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
8 } catch(PDOException $e) {
9     echo "Connection failed: " . $e->getMessage();
10 }
```

A.1.2 Soll-Zustand (Neu: Database.php)

```
1 // includes/Database.php - Singleton Pattern
2 class Database {
3     private static $instance = null;
4     private $pdo;
5
6     private function __construct() {
7         $dsn = "mysql:host=" . DB_HOST . ";dbname=" . DB_NAME;
8         $this->pdo = new PDO($dsn, DB_USER, DB_PASS, $options);
9     }
10
11     public static function getInstance() {
12         if (self::$instance == null) {
13             self::$instance = new Database();
14         }
15         return self::$instance;
16     }
17 }
```

A.2 Datenbank-Singleton (Database.php)

Die zentrale Anbindung an die MariaDB erfolgt über PDO.

```
1 class Database {
2     private static $instance = null;
3     private $pdo;
4
5     private function __construct() {
6         $dsn = "mysql:host=" . DB_HOST . ";port=" . DB_PORT . ";dbname=" . DB_NAME .
7         ";charset=" . DB_CHARSET;
```

```

7     $options = [
8         PDO::ATTR_ERRMODE           => PDO::ERRMODE_EXCEPTION,
9         PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
10        PDO::ATTR_EMULATE_PREPARES  => false,
11    ];
12    $this->pdo = new PDO($dsn, DB_USER, DB_PASS, $options);
13 }
14
15 public static function getInstance() {
16     if (self::$instance === null) {
17         self::$instance = new Database();
18     }
19     return self::$instance;
20 }
21 }

```

A.3 Ticket-System Algorithmus (index.php)

Die Generierung der Ticket-ID und des Geheimworts für das Feedback-System.

```

1 if ($antwort_gewuenscht) {
2     // Generierung einer 6-stelligen ID
3     $ticketId = strtoupper(substr(md5(uniqid(rand(), true)), 0, 6));
4
5     // Auswahl eines Wortes aus einer vordefinierten Liste
6     $wordList = ["Apfel", "Banane", "Croissant", "Gabel", "Holunder", "Kaffee", "Limonade", "Melone", "Nudel"];
7     $geheimwortPlain = $wordList[array_rand($wordList)] . rand(10, 99);
8
9     // Hashing des Geheimworts
10    $geheimwortHash = password_hash($geheimwortPlain, PASSWORD_BCRYPT);
11 }

```

A.4 Ergebnisvisulierung (Logik)

Berechnung der prozentualen Anteile für die Umfrage-Fortschrittsbalken.

```

1 $prozent = ($gesamtAnzahlProTag[$wunschRow['tag']] > 0)
2     ? number_format($abstRow['cnt'] * 100 / $gesamtAnzahlProTag[$wunschRow['tag']], 0)
3     : 0;
4
5 echo "<div class='progress-bg'>";
6 echo "    <div class='progress-bar $colClass' style='width:$prozent%'></div>";
7 echo "</div>";

```

A.5 Design-Variablen (CSS)

Auszug aus der style.css zur Steuerung des Dark Mode Designs.

```

1 :root {
2     --bg-main: #0B0E14;           /* Tiefer dunkler Hintergrund */
3     --bg-card: #1C212D;          /* Karten-Hintergrund */
4     --text-primary: #E2E8F0;      /* Helles Text-Blau/Grau */
5     --accent-color: #3B82F6;      /* Strahlendes Blau */
6     --success-color: #10B981;     /* Smaragdgruen fuer Veggi */
7     --surface-border: rgba(255, 255, 255, 0.08);
8 }

```